

handoff-fanout 系统架构总览

1. 系统定位与运行时拓扑

handoff-fanout 是一套单用户、单台 Mac、内网运行的 macOS CLI / 自动化系统。它服务于一个 owner——这个 owner 同时驱动多个 AI 编码会话（一个监管中枢窗 + 若干 worker 窗），分布在 macOS 的多个桌面（Space）上。系统解决的问题是：把“一个会话干完一段活、交棒给下一个会话”这件事从手工开窗、手工贴 prompt、手工切桌面，变成无人值守的自动接续——并在交棒过程中强制复盘、强制审计、自动把 owner 的视图带到 worker 所在桌面再一步带回原桌面、最后回收死掉的窗口与 git worktree。它不是商业 Web 产品：没有网络监听端口、没有资金流、不托管任何第三方 PII。

系统由五个协作部件构成：

- **handoff CLI** (Python, editable install) —— 交接棒的生产端。handoff dump / handoff spawn / handoff precheck / handoff audit-close / handoff audit-check / handoff watchdog / handoff gc-singlepane 等子命令，由 cli.py 懒加载分派 (cli.py:177)。CLI 本身从不开窗口，只产出“派窗意图”——一组写在 \$HANDOFF_HOME/<project>/ 下的 sidecar 文件。
- **launchd watchdog loop** (Bash + Python) —— 常驻引擎。两条 launchd 入口：install/auto-continue.sh（自动接续 loop，每个 tick 把 ready 的派窗意图变成真实窗口）和 src/handoff_fanout/watchdog.py（周期 backstop，每 60s 跑一遍做心跳监控 / 孤儿清理 / 窗口回收）。
- **VS Code 扩展 dharmaxis.handoff-helper** (TypeScript, 已装 0.6.0) —— 窗口级执行器。在每个冷启动 worktree / singlepane 窗口的扩展宿主里跑，负责单栏折叠、监管接班关前任窗、worker 窗口的回收自闭。所有路径 fail-closed，只能动自己这个窗口。
- **跨仓共享原语** (dharmaxis 仓) —— scripts/vscode-spaces/vscode-spaces.py（桌面定位/跳转/winlist z 序探针）+ code-router.sh（包住真实 code 命令、在开窗前一步把视图滑到目标桌面）。去程 focus-jump 与回程重激活的真实实现都住在这里，本仓只持有调用壳。
- **集团级判别器** (~/.claude/scripts/) —— dx_session_role.py（中枢身份解析）+ dual-brain-runner.py（外双脑审计 runner，产出 *.evidence.json）。

一个交接棒的端到端流向：

```
会话闭环
├─ handoff precheck → 写 precheck/<task>.retro.evidence.json (复盘证据 + 自校验哈希)
├─ handoff dump --retro-evidence FILE
│   ├── STOP 哨兵闸 → 复盘证据闸 (schema/hash/HEAD 新鲜度/Phase D codex 审计)
│   ├── preflight 闸 → worktree / singlepane workspace 解析
│   └─ 写一串 sidecar (先全部 sidecar，最后写 .uri 触发器)
│       ├── queue/<task>.md      人类可读交接棒
│       └─ queue/<task>.uri      launchd WatchPaths 触发器
│                               含 WORKSPACE= / URI= / SPawner_FOCUS= (去程焦点路径)
└─ launchd 看到 .uri 落地 (~1s)
    └─ install/auto-continue.sh (一个 tick)
        ├── _return_precapture      快照 owner 当前桌面 B + 可重激活锚点 (必须在开窗前)
        ├── code -n <workspace>    经 code-router: 先一步跳到 spawner 桌面 A (去程 focus-jump)
        ├── 等目标窗 frontmost (AXRaise 兜底)
        ├── open vscode://...      把 prompt 粘进 Claude 输入框
        ├── 合成 Enter              提交 (以 transcript 增长为准，真提交才置 _RETURN_DISPATCHED=1)
        └─ _return_jump_back        一步把视图重激活回桌面 B (仅真提交才回跳)
worker 会话跑活 → 报告 (touch worker_reported, 不自我 discharge)
├─ 中枢零信任复盘 + 外双脑审交付物
└─ merge / succession
    ├── handoff audit-check (pre-push 硬闸: 缺陷不下传)
    └─ handoff audit-close --coordinator (中枢→中枢交棒, gated dump + 一次性 token 关前任窗)
```

整套系统的状态是文件系统原生的 (filesystem-as-DB)：没有数据库，所有 runtime 真相落在 \$HANDOFF_HOME = ~/.claude-handoff 的目录树里，由 watchdog 与各道闸读写。

2. 技术栈

层	技术	具体情况
CLI 引擎	Python (纯 stdlib, 零运行时依赖)	pyproject.toml 的 dependencies = [] ; 运行时全 stdlib。dev/lint extras : pytest≥8 / pytest-asyncio≥0.23 (地板 pin) / ruff==0.15.5 (精确 pin, 注释说明未 pin 的 ruff 漂红了树)。pip install -e editable, src/ 即 live。uv.lock 在仓。
运行时 loop	Bash (install/auto-continue.sh, 2506 行) + Python (watchdog.py 965 行 / re-claim.py 1540 行 / heartbeat.py 335 行)	set -euo pipefail ; 通用超时包装 run_with_timeout (后台 + kill -0 轮询 + SIGTERM→1s→SIGKILL, 超时 return 124, auto-continue.sh:1079-1096)。
派窗执行器	TypeScript (VS Code 扩展, engines.vscode ^1.85.0)	已装 0.6.0 == 源码 extension/package.json:5, 无版本 skew。零运行时依赖 (dependencies: {}), 全是 devDeps: typescript/esbuild/mocha/@vscode/*), package-lock.json 在仓, main=dist/extension.js。
macOS 原生	Quartz / SkyLight / winlist / CGEvent / AX	winlist (SkyLight z 序前台探针, Swift 编译产物) + Quartz 兜底用于桌面定位; CGEvent HID 注入用于自动解锁登录密码; AX (Accessibility API) 用于 frontmost 判定 + AXRaise + 合成 Enter。本仓不 import pyobjc/Quartz, 全部 shell-out。
版本控制隔离	git worktrees	每个 worker 会话一个独立 git worktree add -b handoff/<task> (worktree.py:1033), 共享 object store、独立 HEAD/index/工作树; 冷启动 worktree 窗的 .handoff.code-workspace 注入。
调度	launchd	两个 plist job: com.dharmaxis.auto-continue (WatchPaths 触发 + 周期) 跑接续 loop; com.handoff-fanout.watchdog.plist (StartInterval 60) 跑周期 watchdog。
外部 CLI (shell-out, host 提供, 未 pin)	osascript (47 refs) / codex (41) / ioreg (10) / caffeinate (9) / vscode-spaces.py (4) / gemini (2) / winlist (1) + dual-brain-runner.py / code / MindPersist venv	这是真正的供应链面——版本随 host, 但每次调用要么 fail-open 要么 env 可覆盖 (测试用), 缺失/变更只降级不损坏。winlist + vscode-spaces.py 住在 dharmaxis、非 pip 安装, 是无 manifest 捕获的跨仓运行时耦合。

3. 子系统

3.1 派会话 DUMP + 复盘证据闸 + PRECHECK

组成: dump.py (2073 行, dump CLI 入口 + 三种交接棒产出 + 七档闸调度 + worktree/singlepane 解析 + 孤儿清理)、retro_gate.py (1507 行, 复盘证据闸核心)、handoff_precheck.py (567 行, precheck CLI + 证据 builder + 规范哈希)、templates.py (703 行, 三种交接棒 markdown 模板 + \$0 审前任 / §-1 复盘 SOP 注入)。

工作机制: handoff_dump 的主流程 (dump.py:1867) 依次过: 参数解析 → --coordinator × 批/fan-in 组合机器拒 (dump.py:1879-1890) → STOP 多层哨兵 (any_stop_auto dump.py:319, 查 done / STOP_AUTO / 项目级/批级四层) → 复盘闸 (_run_retro_gate dump.py:1920) → 批/fan-in 分派 → preflight 闸 (fail-closed dump.py:133) → worktree 解析 (resolve_spawn_workspace, 把后继重定向到独立 worktree 或 BLOCKED 退出) → write_active_dump 在 singlepane 并发硬闸内执行。

write_active_dump (dump.py:812) 的产出顺序由 codex+Gemini 裁定并 test-locked (dump.py:909-916): 所有 sidecar 先写、.uri 触发器最后写 (dump.py:1037-1039, .uri 附 SPawner_FOCUS= 行)。先 <task>.md (crash-atomic) → 终态分支 / active 路径先删旧 BLOCKED.md (防 launcher 跳过有效 .uri, dump.py:907) → ack/<task>.queued → ack/<task>.old_ready (仅当有 retro evidence) → worktree sidecar → singlepane sidecar+workspace → 中枢 memory baseline → pbcopy → 最后 .uri。

复盘闸把“已复盘”从口号变成工具层 invariant。闭环会话先跑 handoff_precheck 把 Phase 0 五项 + Phase 1 五类状态快照成一个自校验哈希的证据文件 (schema_version 5.5.0, handoff_precheck.py:35; 规范 JSON 排序键无空格、排除 hash 字段自身, handoff_precheck.py:189-211)。下一次 dump 的闸 (check_retro_gate retro_gate.py:1226) 三个触发器任一激活即跑: --retro-evidence FILE / HANDOFF_RETRO_BYPASS=1 / HANDOFF_RETRO_MANDATE=1。闸主体依次过 overdue 闸 → bypass 校验 → mandate 路由 → 持有序锁 (precheck→dump) 加载证据并自哈希校验 → phase 状态枚举校验 (非✅ 必带 reason, retro_gate.py:463-501) → HEAD 新鲜度三档 (retro_gate.py:534-608) → 若 audit mandate 开则跑 codex 审计 G0-G9 → 清 attempt 计数器返回。产出七档 exit code (retro_gate.py:56-68): 0=OK / 1=ERR-FATAL (tamper, retry 无用) / 2=ERR-BLOCKED (attempt 耗尽 / head-stale-fatal) / 3=ERR-LOCKED (锁竞争, 让位) / 4=ERR-RETRY (证据缺/hash 不符/schema 不过/HEAD stale) / 6=ERR-BYPASS (bypass 字段缺/follow-up 逾期); exit 5 故意空缺。attempt 状态机封顶 ATTEMPT_MAX=2 (retro_gate.py:91), n≥2 写 BLOCKED.md 硬拒。HEAD 因兄弟会话 commit 漂移时, dump-time re-align (retro_gate.py:1118) 在已持的锁内 CAS-guarded 重绑证据到新 HEAD、拒 ABA reset、不 bump attempt。

现状：单任务 dump (md+uri+sidecar 排序产出) ✓；七档 exit code + stderr 前缀 ✓；复盘证据校验 (schema/hash/枚举/reason) ✓；HEAD 新鲜度三档 + re-align ✓；attempt 状态机 (retro + 隔离 audit 双计数器) ✓；fan-out 批 + fan-in (N_max / 全局上限 / file_ownership Gate A / 错峰 spawn) ✓；worktree 隔离 active 路径 ✓；singlepane 并发硬闸 ✓；孤儿检测+清理 ✓。

缺口：

- `--kill-spawned` flag 黄 — 一名字含“kill”暗示自动关 tab，实现只 print 任务列表 + 发一条 macOS 通知，从不物理关闭任何 VS Code tab (dump.py:1615-1625，自承“IDE tab 标题不带 task_id 做不到”)。
- `--no-dedupe` flag 黄 — 已废弃 no-op，仅向后兼容接受并忽略 (dump.py:1845)，非真功能。
- docs/PROTOCOL.md 反映本子系统现状 ✗ — 文档停在 schema_version 2 / v1.0.0 (2026-05-29)，grep 全文 0 命中 retro/precheck/evidence/mandate/exit/old_ready/overdue/attempt。整个复盘/审计闸链、precheck/ 与 locks/ 目录、worktree/singlepane/中枢概念、.uri 第三行 SPawner_FOCUS= 全无文档。该文档仅可信用于“fan-out 批 + file_ownership Gate A + 原子性保证 + ACK 协议”基础层。
- mandate 生效性缺运行时自证 黄 — HANDOFF_RETRO_MANDATE / HANDOFF_AUDIT_MANDATE 只在 dump 读 env (dump.py:280-281)，代码默认全是 legacy (无 evidence → 放行)。“闸已拨”只存在于会话外的三条 env 注入路径 (.zshenv + launchctl setenv + plist)，任一漂移/未注入则 dump 静默走 legacy 放行，闸形同虚设却无任何告警。闸的存在 ≠ 闸在跑。
- old_ready 写失败只 print 警告、不间断已发布的 dump 黄 — retro evidence 通过但 _write_old_ready 返回 None 时仅 print 一条 (dump.py:948-957)，.uri 仍会发布，新会话 \$0 审前时找不到 old_ready，复盘链在这一棒断开却无人知 (自动接续里没人看 print)。

3.2 CODEX 审计门禁 + 交付审计机器闸 + 中枢交棒

组成：codex_audit.py (2683 行，审计门禁全部 + bypass producer + G0-G9 评估 + audit-close CLI + succession relay)、audit_evidence.py (495 行，交付审计机器闸 audit-check + owner tty 红覆盖 audit-override)、succession_authority.py (220 行，一次性 succession 授权 token)、spawn_nonce.py (23 行，不可猜的 per-spawn 64 位 hex nonce + window.title 装配/校验)。三条相互独立但同源的承重链。

工作机制：

(A) Codex 审计门禁 (dump 时，“缺陷不下传”) — HANDOFF_AUDIT_MANDATE=1 时，retro gate 在 dump 成功前的最后一闸真调 evaluate_audit_gate (retro_gate.py:1446-1481)：会话改了代码却没有一份通过的 codex 审计块，dump 被 RETRY→BLOCKED。审计经四种模式兑现 — full_codex_audit (findings 经 disposition 闭环，G2-G9 全身) / empty_diff_attestation (base..HEAD 无 diff，机器重算为空才放行) / docs_only_light_audit (机器判 diff 全是文档后缀，CLAUDE.md/AGENTS.md/GEMINI.md/prompts/ 强制 full) / codex_unavailable_bypass (≥3 条机器可证 codex 失败 MIN_CODEX_FAILURES=3 + 强制 follow-up，欠债)。audit-close (codex_audit.py:2312) 是把 codex-audit 块折叠进 retro evidence、并在同一把锁下完成 gated dump 的单一入口 — 锁序 precheck.lock → dump.lock → <task>.audit.lock，与 dump gate 同序无死锁。闸不信自报：evidence 里的块只是 producer 自报，gate 对 live HEAD 重算 diff、重读 last run。bypass= 除账：自动写 ack/<task>.audit.override.json (含 follow_up_audit_task_id + 1 天 follow_up_deadline)，两个消费方强制还债 — 逾期扫描器 (auto-continue.sh:2245) 每次调用扫该 sidecar、过期写逾期 marker；下一棒 forced-task 让首个 task 必须是 follow-up，不能在未审代码上继续路线图。

(B) 交付审计机器闸 (push 时，“中枢审派出交付必加外双脑”机器化) — 独立于 (A) 的另一套闸：audit-check (audit_evidence.py:259) 被 .git/hooks/pre-push (硬拒) 和 post-merge (仅 warn + 写 .audit_pending) 调用。它按 head_sha / patch-id (git patch-id --stable) + changed-files 把外双脑 runner 产出的 audits/*.evidence.json 匹配到被推区间。两条匹配路径：head_sha 路径要求 reviewed_head_sha == head_sha 且 reviewed_base_sha 存在且 == base (窄审不得清宽推，audit_evidence.py:162)；patch_id 路径要求 patch-id 一致 + changed-files 集合一致，同 base 还额外要求 diff_sha256 字节级一致 (patch-id 忽略空白，但 Python 缩进即语义，audit_evidence.py:182-186)。verdict fail-closed：任一匹配 RED 且无合法 owner override → FAIL (即便另有 GREEN)；MIXED/ERROR 同样 FAIL (只 audit_unavailable bypass 能开门，绝不清 RED)；唯一放行 = owner 亲手 tty 跑 audit-override (audit_evidence.py:441-446 强制 stdin/stdout 都是 tty，AI 会话无 tty)。

(C) 中枢交棒 (succession relay) — audit-close --coordinator --status active 在内部 gated dump 返回 0 之后 (codex_audit.py:2625)，签发一次性 succession 授权 token (succession_authority.py:107，0600 O_EXCL，文件名↔payload 绑定 + project + successor_task + TTL 120s + 单次 unlink)。它抑制 dump 自己发窗 (suppress_spawn_artifacts)，改由进程内 spawn --role supervisor_succession 消费 token 直接发出继任中枢窗 + 写 queue/<predecessor_task>.done 让共享身份解析器跳过前任陈旧 sidecar + 发一条通知。这是唯一能合法关前任中枢窗的路径，堵住“中枢零复盘交棒”漏洞。spawn 侧 consume_token (spawn.py:603) 绑 successor task、单次 unlink (并发竞争恰一个赢)，绝不失败回退 legacy 自发布 (禁静默降级)。

现状：HANDOFF_AUDIT_MANDATE 门禁真实强制 ✓ (mandate-ON 时真调 G0-G9 并据 outcome 拦 dump)；bypass 欠债逾期扫描器 LIVE 且有 producer ✓；交付审计机器闸 (pre-push 硬拒 / post-merge warn / fail-closed 裁决) ✓；succession 一次性 token 端到端 (issue→consume→unlink→predecessor .done) ✓。

缺口：

- owner_ack_token / audit-override checksum 仅诚实层非加密 — compute_owner_ack_token = sha256(task|finding_hash|nonce|ap-proved_at) (codex_audit.py:167) 与 validate_owner_ack = sha256(reviewed_patch_id|reason|ts) (audit_evidence.py:136) 都是裸 sha256 明文拼接、无密钥。代码注释已诚实标注 (codex_audit.py:140-146)：以 owner 身份运行的 AI 能算出自洽 token + 写 on-disk ack 工件，绕过 G7 / 绕过 RED-override 摩擦。它防的是复用 (finding_hash 绑定) + 永不过期 (7 天 TTL) + 无痕 (jsonl trail + tty 摩擦)，挡 drift 不挡蓄意伪造者。真加密保证需 owner 持私钥 (design deferred)。不能让“有 owner override 闸”掩盖“非加密”。
- mandate_projects 白名单的承重事实 — 闸是否对某项目生效取决于共享 config.json 的一个列表。退化形状 (空/typo/全非法) — 一律 fail-closed 全局强制，但若列表被误填成非空却漏掉某活跃项目，该项目的 mandate 会静默失效 (非退化形状、不触发 fail-closed, dump.py:295-300)。HANDOFF_RETRO_BYPASS / 显式 — retro-evidence 总能绕过白名单上闸。建议运维周期核对漏项。
- 引为 source-of-truth 的 spec 草案文档不存在 — codex_audit.py:21、audit_evidence.py 导言、templates.py 多处把 codex-audit-gate-spec-draft.md 当 SOT 引用，但该文件在当前 HEAD 不存在 (git 无历史)。所有 spec \$X 引用都是悬空引用、无法核对。
- 两处注释把已上线的链说成休眠 (非真休眠) — retro_gate.py:670-672 和 auto-continue.sh:2113-2116 都把 audit-overdue producer 注释成“deferred / dormant-but-ready / 无 audit.override.json 文件存在”，但 producer 已 wired (codex_audit.py:2568)、扫描器已跑、闸已读。代码行为正确，注释落后于实现 — 读注释的人会误判 bypass 是免费午餐。同仓内 codex_audit.py:7-9 docstring 还写“audit mandate OFF (Phase A 唯一状态)”，与 templates.py 的 mandate ON 相矛盾。

3.3 派窗 SPAWN + WORKTREE + FOCUS-JUMP + 锁

组成：spawn.py (761 行, fresh-spawn 意图产出 + 身份/参数校验 + succession-token 收口 + worktree-vs-singlepane 编排 + 焦点解析胶水)、worktree.py (1491 行, per-session worktree 生命周期 + 红顶注入 + 孤儿回收 + GC)、spawner_focus.py (135 行, 去程焦点路径的单一安全闸)、spawn_lock.py (85 行, project_spawn_lock 原子 mkdir 互斥)、config.py (571 行, config.json loader + worker_isolation 路由 + fail-closed parse)。

工作机制：派窗产出“派窗意图” (一组 sidecar)，watchdog 消费后开窗，producer 从不自己开窗。dx-spawn-session.sh --project <全路径> --brief <f> --task-id <id> 是跨项目派窗入口，共享引擎掌 registry worker_isolation 路由 (config.worker_isolation_for config.py:234 返回 worktree | singlepane | None, None ⇒ caller fail-closed, 引擎从不猜隔离模式)。handoff spawn (run_spawn spawn.py:495) 是 fresh-spawn 意图产出器 — 无复盘 mandate 闸 (从不 exit 4, spawn.py:74)、无路线图注入，校验身份 (kebab slug ≤60) 后按 --isolation 分支：worktree → per-session git worktree；singlepane → 出树 .handoff.code-workspace 覆盖真实 repo (不脏化工作树)。

supervisor_succession 角色不是手动路径 — 需 retro-gated audit-close --coordinator --status active 签发的一次性 token，在 produce 前最后一刻消费、绑 THIS task (spawn.py:558-565、spawn.py:600-615)。worktree 创建 (create_worktree worktree.py:821) 在 project_spawn_lock(wait=120.0) 内：mode 解析 → fetch tracking ref → unpushed-HEAD BLOCK 闸 (源 HEAD 未推到 origin、或本地 integration 分支领先 origin 时 BLOCK 不降级, worktree.py:922-941，否则后会从陈旧/丢失的码分支) → collision 分类 → 干净+已发布同 base 的 worktree REUSE 复用分支 (reused=True 避免 rollback 删掉别会话的 worktree) → 否则 git worktree add -b。inject_vscode_workspace (worktree.py:658) 写 .handoff.code-workspace：nonce 绑定的 window.title、单栏 UX keys、是中枢则加 中枢 前缀 + 红 titleBar (worktree.py:775-789)。is_coordinator 从 role == ROLE_SUCCESION 单一来源派生 (spawn.py:431)。MUST-1 fail-closed：若 workspace 标题无法承载本次 nonce (用户跟踪文件 / 写失败)，spawn 拒绝 (spawn.py:449-462)。

去程 focus-jump：spawning 中枢自己的 .handoff.code-workspace 路径被解析、写成 SPAWNER_FOCUS=<path> 行进 .uri。watchdog export 成 \$HANDOFF_SPAWNER_FOCUS，code-router 跑 code <SPAWNER_FOCUS> 在开 worker 前一步把 macOS 滑到 spawner 的桌面 — worker 出生在中枢桌面。解析三档 (spawner_focus.py)：Tier-1 worktree (中枢 cwd 即其 worktree, <cwd>/handoff.code-workspace, 无 flag 自识别) / Tier-2 singlepane (中枢 cwd 是共享 repo 根、自报 --self-task 重建 <home>/<project>/singlepane/<self-task>.handoff.code-workspace) / Tier-0 env (\$HANDOFF_WINDOW_FOCUS_PATH, legacy)。每个候选都被 validate_spawner_focus 复验 (realpath / 后缀 .handoff.code-workspace / isfile / 在允许根下)，spawn 与 dump 共享这一个安全边界。

锁并发契约：project_spawn_lock (spawn_lock.py:24) 原子 mkdir 获取 (macOS 无可移植 flock)，TTL 默认 120s, wait=0.0 默认非阻塞。singlepane 用默认 wait=0.0 → 竞争即抛 LockHeld (\$5.4 硬拒 + watchdog skip 瞬发)；并行 worktree worker 传 wait=120.0 (合法并发 — 它们改同一源 repo 的 .git/config/tracking refs, 所以排队不拒)。两 worker 竞争破同一陈旧锁时，输者 re-mkdir 抛 FileNotFoundError → 重审 → 对手 FRESH 锁给干净 LockHeld, churn 由 max_stale_breaks=5 封顶, always-release in finally。

现状：worktree + singlepane 产出器 (fail-closed / lock-serialized / 带 rollback) ✓；红顶中枢标记 (fresh-create + reuse 双路径 + 失败非静默 WARN) ✓；spawn_lock 原子互斥 (crash-free race + 封顶 stale-break + always-release) ✓；unified_spawn_enabled / worker_isolation fail-closed config ✓；SPAWNER_FOCUS 去程在 dump 与 spawn 两条路径都接线 ✓；Tier-1 + Tier-2 self-identification (env-independent) ✓。

缺口：

- env-path 通道 \$HANDOFF_WINDOW_FOCUS_PATH — 注入进每个引擎 workspace 的 terminal.integrated.env.osx (worktree.py:534-557)，但对扩展自动派生的 singlepane 中枢不到达其 agent shell (在 agent shell 里为空)。Tier-1/2 自识别正是为绕过这条死通道而加的。残留陷阱：singlepane 中枢若忘传 --self-task，回落 fail-open (worker 照样 spawn，但不跳桌面、静默 UX 降级)。这是 by-design fail-open (UX hint 不该阻断 spawn)。

- `add_worktree_or_reclaim_orphan` 是未接线的并行原语 🟡 ——有完整孤儿回收逻辑 + 显式红顶 INVARIANT 警告 (`worktree.py:1118-1196`)，但不被 `create_worktree` 调用 (live add 路径是 `inline git worktree add`)。今天无 bug (未接线)，但是 latent footgun：未来若把 add 路由经它却不重调 `inject_vscode_workspace`，会开出一个无红顶的回收中枢 `worktree`，静默破“只要是中枢窗口就必须红顶”。
- `lock heartbeat / mtime refresh` ❌ ——见下文 §3 已知 P1。

已知 P1 (确认真实)：`project_spawn_lock` 的锁目录 `mtime` 在获取时设定、之后从不刷新 (无 `heartbeat`, `spawn_lock.py:47`)，`ttl=120s`。waiter 纯按 `age vs ttl` 判陈旧、无 **liveness 检查** (无 PID 探测, `spawn_lock.py:59`)。`create_worktree` 做网络 `git fetch` (30s 超时) + `git worktree add` (60s 超时) + 多个 `git` 调用——慢远端/大 repo 下临界区可超 120s。而 `worktree caller` 故意用 `wait=120.0` (`spawn.py:393`) 让排队 worker 在 TTL 边界附近正活跃地轮询：它见 `age ≥ 120` 就 `rmdir` 仍被持有的锁、`mkdir` 自己的——原持有者还在 `create_worktree` 内、worker #2 已进自己临界区，两者并发改同一源 repo 的 `.git/config/tracking refs` (“could not lock config file”/index-clash 类，正是锁要防的)。释放时原持有者 `finally` 里 `rmdir` 可能删掉 worker #2 新 `mkdir` 的锁 (同路径不同 inode)，`suppression` 隐藏了 `cross-delete`。缓解方向 (未在范围内实施)：`held-lock heartbeat / stale-break` 前 PID `liveness` / 把 `ttl` 提到大于 `create_worktree` 最坏墙钟 (如 300s)。

3.4 运行时 WATCHDOG + 自动接续 + 心跳 + §6C RECLAIM + GC

组成：`install/auto-continue.sh` (2506 行, `launchd` 自动接续 loop)、`watchdog.py` (965 行, 周期 6-mode backstop + orphan 分流 + §6c reclaim tick 挂载)、`reclaim.py` (1540 行, §6c worker `worktree` 窗口回收 contract)、`heartbeat.py` (335 行, fan-in tab 伴生 `heartbeat daemon + metrics`)、`gc_singlepane.py` (295 行, STALE `singlepane` 中枢 `sidecar` 卫生 janitor)。去程/回程的真实原语住在跨仓 `dharmaxis vscode-spaces.py`, `auto-continue.sh` 经 `$(dirname $HANDOFF_CODE_BIN)/vscode-spaces.py` 调用 (`auto-continue.sh:1308`)。

工作机制：自动接续 loop 每个 `launchd tick` 遍历各项目 `queue/`，发现 `ready` 的 `.uri` (去 `lock` → `caffeinate` 防睡 → 可选自动解锁) → 解析 `.uri` 三行 (`WORKSPACE= / URI= /` 可选 `SPAWNER_FOCUS=`, `auto-continue.sh:1413-1423`) → `_return_precapture` 在开窗前快照 `owner` 桌面 B (`auto-continue.sh:1665`, 此时 `frontmost` 未被子窗污染) → `code -n <workspace>` (经 `code-router`, 去程 `focus-jump` 在它内部按 `HANDOFF_SPAWNER_FOCUS` 发生) → 等目标窗 `frontmost` (真墙钟封顶 3s + `AXRaise` 兜底 + 一次“刚开的冷渲染滞后标题”判别人, `auto-continue.sh:1722-1730`) → `open URI` 把 `prompt` 粘进 → 提交闸 (`screen_is_locked` 必 `rc=1` → `accessibility_trusted` → 合成 `Enter`, `title-nonce + readiness gate`, 真提交以 `transcript` 增长为准) → 提交成功才 `_RETURN_DISPATCHED=1` → `_return_jump_back` 一步把视图重激活回桌面 B (`auto-continue.sh:2006`, **正向门控**：所有未确认 `Enter` 路径都不回跳, `owner` 永不在 `worker` 未提交时被拽回)。

`watchdog tick` (`watchdog.py:914`) 在 `O_EXCL` 锁 (`stale>30min` 自清, `watchdog.py:90`) 下做 6 个 `backstop mode + orphan` 扫 + 单任务心跳扫 + §6c reclaim tick：Mode 1-3 fan-in 批次 (完成未触发 / fan-in 心跳僵死 / 超时降级)；Mode 4 + Mode 6 = **529 防御** (子任务/单任务心跳 >5min 无更新 → 写 `.529-suspected` + 通知 + `SIGTERM`→5s→`SIGKILL` 杀 `wedged` 进程, `watchdog.py:427`, 杀后再探问“假 killed”, `literal heartbeat` 路径正则 `escape` 防误杀同名任务/测试 runner)；Mode 5 (`scan_orphan_spawns` `watchdog.py:768`) 跨项目扫 `ack/*.spawned` 但 `.md` 消失的孤儿, 用 `.singlepane sidecar` 的 `isolation` 字段区分 `spawn(有)` vs `dump(无)`：ACTIVE 静默跳过 / STALE 写 `.stale-spawn` (非 BLOCKED.md, 不误导 `owner` 关活窗) / legacy dump 残留逐字节保留旧判定。

§6c window reclaim (`reclaim.py`) 是哨兵驱动的 worker 窗口/worktree 回收 (PULL 模型)：record-head 记被合并的 SHA → `coordinator` 写 `ack/<id>,reclaim_requested` → `watchdog tick N` 在项目 `.spawn.lock` 下经 `transcript-mtime` 活性探针 `fail-CLOSED` (读异常 ⇒ 无条件 alive ⇒ 永不关, `reclaim.py:261-287`) → 合法则写 `reclaim_pending.json` 授权 → 目标窗扩展自己轮询自闭、回写 `close_issued` `ack` → `tick N+1` 读 `host_pid.json` 调 `_host_pid_liveness` (`os.kill(pid,0)` `ESRCH=dead` 才删 `worktree`, `reclaim.py:882`, PID 复用 `fail-closed`)。这是 PID dead-man：producer 凭宿主真死才回收 `worktree`。

现状：`watchdog 6-mode backstop + 529 防御` ✅ (mode 1-6 全活, 杀 `wedged` 进程上线, `watchdog.log mtime` 活跃)；自动接续 `dispatch + Enter` 提交 (`cold/singlepane/warm` 三路径) ✅；去程 `focus-jump` (显式 `--self-task` 路径) **LIVE 已观测** ✅ (`~/vscode-spaces/router.log` 35 条 `focus-jump`：✅ 直跳 N-M, 含 `spawner=...handoff-fanout/singlepane/...` 的真实派窗一步落中枢桌面；另 35 条 `focus-jump OK` → 跳过 `goto`, 对照 145 条 `project-goto`)；回程 `spawn-return` 一步回原点 **LIVE 已观测** ✅ (`auto-continue.log` 31 次 `spawn-return` 事件：10 次 `RETURN-REACTIVATE-WS` 一步重激活 + 2 次 `owner` 漂移则不回跳的 `RETURN-ABANDON` 保护)；`heartbeat daemon + metrics/calibration` ✅。注：`focus-jump` 的自动派生 `--self-task` (中枢忘传时由身份解析器补) 需先 GC 收敛 backlog 到唯一 `active` 才激活——见缺口与 `gap-analysis`。

缺口：

- §6c reclaim 哨兵驱动、当前 IDLE (非死、就绪待触发) 🟡 ——producer (`watchdog tick`) 已接线、代码全活 (PID dead-man / pending 状态机 / 19-reason 失败枚举)，但 `tick` 是哨兵触发，无 `reclaim_requested` 哨兵的 `tick` 是 no-op。实地核查盘上当前零 reclaim 哨兵 = 此刻无在飞回收。声称端到端 LIVE 须有一次真 `coordinator`→扩展→PID-ESRCH→`worktree` 删的证据，仅 `tick` 接线不够 (producer 侧 ≠ E2E)。
- `gc-singlepane` 无自动周期 🟡 ——`sidecar` 积压 (同一 `cwd` 多 `active` 让共享 `identity resolver` 歧义) 是 `gc janitor` 排空的对象，但它不在任何 `launchd` 间隔里、默认 `dry-run`，靠人记得手动 `handoff gc-singlepane --execute`。设计即“one-shot + ongoing 手动 hygiene” (避免误删活窗的 `always-on` 风险)，但应在 `runbook/月度蒸馏登记` 定期手动 GC。

- 去程/回程真实原语跨仓 —— `auto-continue.sh` 的 `_return_precapture / _return_jump_back` 只是壳，真活全在跨仓 `vscode-spaces.py`。 `$CODE_BIN` 不指 `router`（如裸 `code`）→ 回程整体静默 `disarm`（`fail-open`）。运维上“回程没生效”可能只是 `HANDOFF_CODE_BIN` 未指 `router`，排查须先验它。

已知 P1（确认真实，根因比预期更尖锐）：`_return_precapture`（`auto-continue.sh:1326`）和 `_return_jump_back`（`auto-continue.sh:1351`）都同步裸调 `/usr/bin/python3 vscode-spaces.py ...`，没有任何 `run_with_timeout` 包装（仓内确有此通用包装、`lock-probe` 路径已用它，回程两函数未享用）。`vscode-spaces.py` 内部的 `subprocess.run(timeout=...)` 只覆盖单个子调用，不覆盖 `ensure_wishlist()` 里的 `swiftc` 编译（跨仓 `vscode-spaces.py:82`，无 `timeout`）——首次运行 `/wishlist.swift` 更新 / NFS 抖动 / `swiftc` 卡住时这一步可无限阻塞。该进程挂在自动接续 `loop` 的同步主路径上，一旦卡死 → `iteration` 永不返回 → 每任务 `lock` 不释放、`caffeinate` 不停、后续 `spawn` 全阻塞。缓解项（`spawn-return` 恒 `exit 0 fail-open`）只在进程能跑到退出时成立，卡在无 `timeout` 的 `swiftc` 时进程根本到不了 `sys.exit(0)`。当前 `wishlist binary` 已编译、happy path 不触发 `swiftc`，是“侥幸”非结构保证。正解：用既有 `run_with_timeout` 包两处调用（超时 `rc=124` 按 `fail-open` 处理），并给 `swiftc` 编译加 `timeout`。

3.5 VS CODE 扩展 + 状态板 + 原子原语 + 安装器

组成：`extension/src/extension.ts`（331 行，激活胶水 + `UriHandler` + `reclaim` 轮询器 + `startup` 单栏 + `ack/host_pid` 落字节 + `closeWindow`）、`handoffReclaim.ts`（431 行，`$6c reclaim` 七 `gate` 决策核 + `A-poll pull` + 关窗二次脏检）、`handoffClose.ts`（444 行，纯 `vscode-free` 关闭逻辑：URI 解析 / `nonce` 格式校验 / `dirty-safe` `retry-once` 关闭 / `succession` `autoclose` / 单栏折叠）、`status_board.py`（1643 行，只读状态板）、`atomic.py`（303 行，原子写 + `flock` 目录锁）、`safe_commit.py`（238 行，防 `index` 劫持 `commit` 包装）、`prune.py`（111 行，死 `sidecar` 清理）、`memory_baseline.py`（267 行，G3 沉淀证明）、`install/install.sh`（293 行，幂等安装器）。

工作机制：扩展是冷启动窗口的窗口级执行器，对本窗口做三件 `fail-closed` 的事。URI 契约注册 `vscode://dharmaxis.handoff-helper/<path>?<query>`（`extension.ts:227-313`）：`/singlepane`（折叠侧边栏成单编辑窗）/`/autoclose`（角色门控的监管接班前任窗 + 兼容残留的 `reclaim` 接收口）/ 未知 `path` 静默忽略。窗口自定位靠 `title` 子串匹配（本窗 `window.title` 是否含该 16-hex `nonce`，`handoffClose.ts:266-268`）——一个 `vscode://` URI 只落到一个窗口的扩展宿主、宿主只能动自己的窗口，所以“错投的 URI 永远关不了错的窗口”是 `fail-closed` 内禀属性。脏页保护：任何关闭路径先过脏页门，`succession` 是 `skip-dirty`，`$6c reclaim` 更严——任一脏 `tab` 即拒整次关闭并写 `dirty ack`，关窗前还二次重扫（`gate` 与 `closeWindow` 之间可能冒新脏 `tab`，`handoffReclaim.ts:421-430`）。红顶中枢窗永不是 `$6c reclaim` 目标（走 `succession` 退场）。

`$6c reclaim` 七 `gate` 链（`handleReclaim` `handoffReclaim.ts:156-275`，拒绝先于任何副作用）：`slug` 校验 → `rolexreason` 白名单矩阵 → `nonce` 必 `hex16`（`nonce` 即 `auth token`）→ `run_id/issued_at/ack_timeout freshness`（超 `issued_at+ack_timeout` 窗口 → `close-command-expired`，防陈旧 URI 杀掉新 `spawn` 占的窗口）→ 窗口本地自定位（`title` 不含 `nonce` → 静默）→ 脏页门 → `dirty-safe` `retry-once` 关 `tab` → `close_issued` `ack` 意图（非终态）。A-poll pull 模型：根因是 `open vscode://...` 只投递到一个焦点窗口、跨桌面 `worker` 收不到 → `producer` 总 `ack-timeout`；修法 `push→pull`——`producer` 写 `reclaim_pending.json`，本窗口每 7s 轮询自己 `task` 的 `pending`（`extension.ts:33`）、重建 `close params` 跑同一不变决策核，窗口定位变成内禀。PID dead-man：窗口 `activate` 时把自己扩展宿主 `pid` 写进 `ack/<task>.host_pid.json`（`extension.ts:65-89`），`producer` 关窗后 `os.kill(pid,0)` 确认宿主真死才回收 `worktree`。

原子原语：`atomic.py` 提供 `atomic_create`（`O_CREATIO_EXCL`）、`write_with_fsync`（`O_TRUNC` 就地写，有撕裂读窗口）、`atomic_replace`（`same-dir temp + fsync + os.replace`，读者只见全旧或全新，`hash` 校验产物必用，`atomic.py:63-94`）、`acquire_dir_lock`（`flock` 跨进程互斥，内核为 `fencing` 权威、持锁进程死时自动释放、无 `staleness` 启发、`re-entrant` 用 `fd` 注册表深度计数、遇 `legacy mkdir` `lockdir` 抛 `LockMigrationError` `fail-closed`、活着但 `hung` 的持有者永不强破）。`safe_commit.py` 是 4 层防 `git-index` 劫持的 `commit` 包装（跨进程锁 + `staged-set` 期望文件不变量 + `pre-commit` `hook` 复验 + `post-audit` `git show --stat` 落地集 $\subseteq$ `expected`）。

现状：`$6c reclaim` + `PID dead-man` 代码 live、回收 E2E 从未成功（扩展 0.6.0 已装=源码一致、`A-poll pull` 接线完整）。精确水位（实地坐实，勿过度声称也勿冤枉）：① `host_pid` 写入侧已在生产中走过——盘上 21 个真实任务的 `*.host_pid.json`（`focusjump-fix / mp-lr-return-fix / dump-lock-fix` 等，扩展 `activate` 即写 `pid`），证明 `PID dead-man` 的“写”端真跑过；② 但完整回收 E2E 从未成功——0 `reclaim_done` / 0 `close_issued`；记录在案的仅 3 次 `reclaim_failed`（2026-06-11），全是 `ack-timeout`，且是 `A-poll pull` 改造之前的旧 `push` 模型失败（`pull` 模型正是为修这个 `ack-timeout` 而建，见下文 `A-poll`）；当前 `pull` 模型下的成功 `coordinator→close_issued→PID-ESRCH→worktree` 删 仍无一次证据。结论与 §3.4 / GAP A4 一致：扩展已装 ≠ `reclaim` 经实战。单栏折叠 live（`onStartupFinished` + `/singlepane` URI 双路，`.hand-off.code-workspace` 守卫）；`nonce` 自定位 live；`atomic/flock/safe-commit` 原语 live。

缺口：

- `install.sh` 与 live 扩展意图直接矛盾（最严重 ✗）—— `install/install.sh:252-267` 步骤 5 标题写“handoff-helper VS Code extension — REMOVED (autoclose feature dropped)”，正文称扩展 `obsolete`、“existed ONLY to drive autoclose”，并主动 `code --uninstall-extension dharmaxis.handoff-helper`（`uninstall` 路径同样卸它）。但 live 扩展是 0.6.0、含完整 `singlepane` + `$6c reclaim` + `PID dead-man`。任何人跑一次 `install/install.sh`（不带 `--no-extension`）会卸掉正在运行的 0.6.0 扩展，导致冷启动窗口不再单栏折叠、`$6c worktree` 回收彻底失效、`succession` 关窗失效。这是注释/迁移逻辑滞后实现约两周的活坑——step 5 还停在“autoclose 被砍”的旧世界观，而单栏折叠与 `$6c reclaim` 都重新建在这个扩展上。在修之前任何人跑 `install.sh` 必须 `--no-extension`。

- vsix 产物版本错配 / 命名不一致 🟡 —— `extension/` 里有 `handoff-helper-0.4.0.vsix` (旧) + `handoff-helper.vsix` (无版本号, `package.json` 已 0.6.0)。手动装若选到 0.4.0 会装到无 `$6c reclaim/PID dead-man/A-poll` 的旧版, `worker` 窗口收不到 `reclaim`、`worktree` 不回收。
- 状态板 `status_board.py` 半接入 🟡 —— CLI 子命令 (`status/sessions/stop/pause/resume/approve/force-sync`) 可用, 但未接入运行引擎 (`dump / worktree / audit-close` 从不 import 它, `status_board.py:55-58`)。设计如此 (只读观测层、“只增不改运行路径”红线), 它把每个 `task` 归一成业务维度 (运行中/卡住需介入/已交付待审/已交付可关/闲置/已完成), 脑裂时真实 `runtime` 赢。
- succession autoclose 默认 OFF 🟡 —— opt-in 三路 (`HANDOFF_AUTOCLOSE_ENABLED=1` `env / ~/.claude-handoff/autoclose.enabled / per-project sentinel`) 实查全部未拨。旧中枢窗不自动关 (owner 手关) 是有意的默认安全姿态 (误关丢 `context`), 非缺陷。 `handleHandoffClose` (`handoffClose.ts:129-148`) 整个函数注释自承是已死的粗粒度关闭原语、仅作测试面留存。
- `memory_baseline G3` 验证 `WARN-only` 🟡 —— 协调器派发时快照项目 `memory *.md` 的 `sha256` 基线、`relay` 时对比 (≥ 1 文件新增/改 = 物理沉淀证据), 但当前从不阻断 `relay` (`observe-then-enforce`), 硬 `enforce` 待 owner 拍板。
- `nonce` 校验是 `coarse` 子串、非 `ack` 内容比对 🟡 —— `title.includes(nonce)` 而非把 `URI nonce` 与 `ack` 内容核对。16-hex 熵让子串假阳天文级低, 是可接受的已知简化, 非活跃 bug。
- `install.sh` 不安装扩展 ❌ —— 它只卸载 `handoff-helper`, 扩展靠手动 `code --install-extension` 装。

部署陷阱 (承重): `install/git-hooks/post-commit` 在 `commit` 后若触及 `install/auto-continue.sh` 或 `install/dump-handoff.py`, 自动 `bash install.sh --sync-launcher / --sync-dump` 把刚提交版本推到 `live launcher` 运行时副本 (`~/.local/bin/`)。在 `worktree` 内 `commit` 这两个 `asset` 会自动改 `live launcher` —— 改 `watchdog/install` 的会话 `commit` 前必 `export HANDOFF_INSTALL_SH=/nonexistent` (`hook [ -f "$installer" ]` 假则跳过), `commit` 后验 `live sha` 未变; `src/*.py` 不匹配则不触发。自动部署还要求匹配的 `dual-brain audit evidence` (无 `evidence` → 响铃 `WARN` + 跳过 `sync`, `fail-closed` 但仍 `exit 0`, 从不 `fail commit`)。人工补救 `install.sh --sync-*` 不走 `gate` (保留最后 `live` 一步在人手手上)。

4. 数据流 / 状态流

所有持久状态在 `$HANDOFF_HOME = ~/.claude-handoff` (`config.home_dir()`), 文件系统即 `runtime` 真相。每个项目一棵子树, 外加全局哨兵。

目录与谁读写:

handoff-fanout 系统架构总览

目录 / 文件	写者	读者	角色
queue/<task>.md	dump (atomic_re-place)	人 / 新会话	人类可读交接棒
queue/<task>.uri	dump/spawn (最后写, atomic_re-place)	launchd WatchPaths → auto-continue.sh	派窗触发器, 三行 WORKSPACE= / URI= / SPAWNER_F WORKSPACE 在 worktrees/* ⇒ repo ⇒ SINGLEPANE 路径
queue/<task>.{done,BLOCKED.md}	dump / retro_gate / watchdog	watchdog / status_board	终态标记
queue/<task>.heartbeat	会话每 60s touch	watchdog Mode 4/6 / worktree GC	<600s 新鲜 = LIVE worktree, 永不
queue/<task>.529-suspected	watchdog	人	心跳僵死 = 疑似 529
queue/<task>.stale-spawn	watchdog Mode 5 (雾 等 size-checked)	人	孤儿 spawn note (非 BLOCKED, 雾)
queue/<task>.singlepane	spawn (带 isola-tion) / dump (无 isolation)	watchdog Mode 5 分流 / gc-sin-glepane / succes-sion relay 路由探针	紧凑单行 JSON : workspace/role/close_policy/spawn, decessor_nonce
singlepane/<task>.handoff.code-workspace	spawn _produce_s-inglepane / dump	VS Code (watch-dog 开)	出树 workspace, folders→真 repo, focus 目标
worktrees/<task>/ + 分支 handoff/<task>	create_worktree	会话 / GC	隔离 git 树/index/HEAD
worktrees/<task>/.handoff.code-workspace	inject_vscode_-workspace	watchdog (COLD) / VS Code / 扩展	nonce title + red-top (中枢时)
ack/<task>.queued	dump	watchdog	dump 已跑待 spawn 面包屑
ack/<task>.old_ready	dump (仅有 retro evidence)	新会话 \$0 审前任	审计元数据 : retro_evidence_hash / next_session_forced_task / code_re
ack/<task>.worktree	dump	find_re-claimable / GC	worktree 回收元数据
ack/<task>. {spawned,submitted,failed,worker_reported}	watcher / worker	orphan 扫描 / status_board	launcher ACK + worker 报告哨兵
ack/<task>.retro. {attempt_n.txt,retry_audit.jsonl,warnings.txt,over-ride.json}	retro_gate / auto-continue.sh	retro_gate	复盘 attempt 状态机 + bypass overr
ack/<task>.audit. {attempt_n.txt,override.json,retry_audit.jsonl}	codex_audit / retro_gate	retro_gate / 逾期扫 描器	隔离的审计计数器 + bypass 欠债 sic
ack/<task>.{retro_overdue,audit_overdue}.txt	auto-continue.sh 逾期 扫描器	闸	逾期债 marker
ack/<task>.reclaim_requested	coordinator	reclaim tick	\$6c 回收哨兵 (JSON run_id/ts)
ack/<task>.reclaim_pending.json	reclaim producer tick N	扩展轮询 + produc-er tick N+1	回收授权 (role/reason/nonce/run_id/issued_
ack/<task>.host_pid.json	扩展 activate (temp+rename)	_host_pid_live-ness	PID dead-man token (pid/nonce/pr
ack/<task>. {reclaim_ack,reclaim_done,reclaim_failed}.json	扩展 / producer	producer / 人	close_issued ack + 终态 (19-reaso
ack/<task>.autoclose_{failed.txt,done}	auto-continue.sh try_autoclose	防 loop	succession 关窗结果

目录 / 文件	写者	读者	角色
precheck/<task>.retro.evidence.json	precheck (atomic+fsync)	dump 复盘闸	复盘证据 (schema 5.5.0 + session, 状态 + 自校验 hash)
audits/*.evidence.json	外双脑 runner	audit-check	交付审计证据 (head_sha/patch-id/diff_sha256/changed_files/verdict)
audits/bypasses/*.json	owner 应急	audit-check	一次性 audit_unavailable bypass 留 used_at/used_range)
audits/.audit_pending	post-merge FAIL 时写 / PASS 时清	人	待补审 marker
authority/succession-<task>.<nonce>.token	issue_token (0600 O_EXCL)	consume_token (unlink)	一次性中枢交棒授权
authority/succession-audit.log	succession relay	forensics	ISSUED/CONSUMED/REJECTED/ (best-effort, 非闸)
locks/precheck.lock / locks/dump.lock / <task>.{retro,audit}.attempt.lock / <task>.audit.lock	retro_gate	retro_gate	有序锁 (precheck→dump→audit)
<project>/.spawn.lock (+ .reclaim-owner.json)	spawn-intent producers / reclaim	spawn / dump / autotclose / reclaim	项目级原子 mkdir 互斥
_gc_quarantine/<stamp>/<相对路径>	gc-singlepane _quarantine (shutil.move, 可逆)	人 (可逆恢复)	隔离区
batches/<batch>/manifest.json + <sub>.env + 各哨兵	dump fan-out / heartbeat	watchdog Mode 1-3 / fan-in	批 + fan-in 状态
launched/<task>--<ts>.txt	launcher	orphan 清理	launcher 落点记录
全局: done / STOP_AUTO / <project>/STOP_AUTO / batches//STOP	人 / 系统	any_stop_auto	急停哨兵
metrics.jsonl	heartbeat (O_APPEND+fsync)	calibration reader	任务工时校准 (非系统遥测)
auto-continue.log / watchdog.log / ~/.vscode-spaces/router.log	各 loop	人	平文本日志 (无 rotation)

crash-atomic 写契约：所有持久写经 atomic_create (O_CREATIO_EXCL, race-safe) / atomic_replace (same-dir temp + fsync + os-replace, 读者只见全旧或全新, hash 校验产物必用) / write_with_fsync ——三者都 fsync 文件和父目录以保 power-loss durability (atomic.py:7,28-94)。状态文件必须在本地磁盘：NFS/SMB/FUSE 破坏原子性保证 (atomic.py:9-11)。crash 的锁持有者由内核自动释放 (flock)。`.uri` 触发器最后写、所有 sidecar 先写——这个排序保证 launchd 看到 `.uri` 时所有依赖 sidecar 都已落盘。

5. 运维与非功能性 (NFR) 横切体检

商业 NFR (支付 / GDPR / DSAR / WCAG / SOC2 / i18n) = N/A (单用户单机内网工具, 无网络监听 / 无资金 / 无第三方 PII; 唯一“个人数据”是 owner 自己磁盘上自己的 task 文本/transcript; 唯一“accessibility”代码是 keystroke 注入用的 macOS Accessibility API 权限, 非面向用户的 a11y)。

以下按这类内网自动化工具实际需要的维度逐项三态评分。

1. 部署 / install 安全 — **半成品** install.sh 本身硬化 (幂等 / set -euo pipefail / 支持 --uninstall / 动作前校验 asset-dir 布局 / curl-pipe 模式 trap rm 清理)。但 launcher 不是 editable/symlink、而是**部署副本**—— ~/.local/bin/auto-continue.sh 与 dump-handoff.py 是 byte-copy, 只靠 install.sh --sync-launcher / --sync-dump 保持当前 (install.sh:84-133)。Python 包是 editable (src/ 即 live), shell launcher 不是, 这个不对称是常驻部署隐患。两道 backstop 补漏: post-commit hook 自动 sync (但本身被 audit-check gate, brain-down 时 fail-closed 跳过部署却仍 exit 0) + 启动 drift guard (比对 canonical sha)。**承重 footgun**: ff-merge 不触发 post-commit → 带入 launcher 变更的快速合并不自动部署, runbook 要求手动 --sync-launcher + launchctl kickstart -k 清 drift。

2. 可观测性 / 日志 — 🟡 半成品 到处有日志，但全是平文本 **append-only**、无 **rotation**，无 **metrics/traces**。 `auto-continue.log` / `watchdog.log`（每 60s 跑、`append-forever`） / `router.log` 都无 **size cap**。唯一结构化诊断通道 `coord-identity-diag.log` **5MB 封顶后停写不轮转、写失败静默吞**（`dx_session_role.py:46,66`）——这是头号可观测性陷阱：撞 **cap** 后每个后续的模糊中枢身份事件（正是你想 debug 的那个）都不可见。有一条窄 **metrics** 通道（`fan-in` 写 `metrics.jsonl`，**O_APPEND+fsync**），但那是工时校准非系统遥测。**Traces** ❌ 完全没有（无 **OpenTelemetry** / **span** / **correlation ID**）。 `audits/*.evidence.json` 是最接近“审了什么”结构化事件日志的东西（ **durable**、 **hash-bound**、 **阅读**）。

3. 凭据 / **unlock** 安全面 — 🟡 （机制安全，但有一条带凭据的旁路 **LIVE**）凭据设计正确（仅 **Keychain**、无 **on-disk** 密码、**shell-out** 隔离），但最高风险路径（**CGEvent** 登录密码注入）此刻对 **erp-system** 拨 **ON**。解锁路径 = **CGEvent** **HID** 注入 **Mac** 登录密码，密码从 **Keychain** **mindpersist-login-password** 读（实测该 **genp item** 存在）；**handoff-fanout** 从不把密码写盘（**grep** 实证 **src/** 零密码落盘，只有 **succession token** 是 **0600/120s** 临时 **nonce**）。本仓不 **import pyobjc/Quartz**、**shell-out** 到 **MindPersist** **venv**。 **opt-in** 是唯一启用器——**per-project** `<project>/unlock.enabled` 哨兵（全局 **env** 启用器被故意移除以防一个 **stray export** 到处武装密码注入）；实测 `~/claude-handoff/erp-system/unlock.enabled` 存在 → 仅 **erp-system** 武装。三道并发/安全护栏全接线（全局 `.unlock.lock` 互斥 / 错密码 **N=2** 后 **cooldown** 防 **macOS** 账户锁定 **retry** 风暴 / **caffeinate** + **Enter** 前再探）。爆炸半径 = **Mac** 登录密码（全本地账户），缓解：只在 **Keychain**（**OS** 保护）、只在自动解锁跑可见会话时注入、默认 **off**。绝不把 **Live-DB** / **root** / **SSH** 凭据下放给 **AI**——闸把最后 **live** 一步留在人的钥匙上。**owner** 自觉接受了这一物理风险类。

4. 失败模式 — ✅ 有失败模式显式设计，刻意分裂：路由/**UX** 路径 **fail-OPEN**（永不阻断开窗），正确性/安全路径 **fail-CLOSED**（`auto-continue.sh` 有 11 处 **fail-open** + 15 处 **fail-closed** 标注）。**launchd** 不跑 → 静默 **stall** 非损坏（**watchdog** 是独立 **job** 仍扫）。**VS Code/** `code` 缺 → 延后 `.uri` 不认领。桌面探针（**winlist/Quartz**）不可用 → **code-router** 零侵入 **fail-open**（直 **exec** 真 **code** 不路由）。锁探测不可靠（现代 **macOS** **ioreg**）→ 响亮 **WARN** + 拒信 **ioreg** → **UNKNOWN** → 延后 **fail-closed**。**codex/gemini** 宕 → 审计 **verdict** 走 **ERROR** **fail-closed**（**push** 拒），唯一门 = **owner** **tty override**。磁盘满/断电 → 原子写保证读者只见全旧或全新。网络断 → **push** 失败本地，**pre-push** 闸在网络前跑、**brain-down** 阻断 **push** 尝试本身。**Caveat** : **alive-but-hung** 的锁持有者永不强破（交给 **watchdog**/超时），**hung** 持有者能 **wedge** 一个项目的 **spawn** 直到 **watchdog** 介入。

5. 状态持久化 / 备份 — 🟡 半成品 每个写都 **crash-atomic**、协议本身文件系统原生，但 **24GB** 状态树**无应用级备份/恢复**，且若干哨兵类无限累积。所有持久状态在 `~/claude-handoff`（实测 **24GB** / **102** 顶层条目），写经 **atomic_create/atomic_replace/write_with_fsync**（全 **fsync** 文件 + 父目录）。文件系统树就是运行真相（**queue/ack/audits** 驱动 **watchdog** 与 闸）；**org-memory** 是协调真相非 **runtime** 状态，是不同层。❌ **无应用级备份/恢复**： `~/claude-handoff` 无版本控制（无 `.git`）、无 **export/restore**；它被 **Time-Machine** **Included**（读起来像“已备份”），但若目录丢失而 **TM** 不是 **current**，在飞 **handoff**（**queue/uri**、延后 **marker**、审计证据、**succession token**）就没了、无恢复路径，恢复故事 = “从头重 **spawn**”。🟡 哨兵无限累积、多数类无 **GC**： `prune.py` 只调和 **queue** `*.done` / `*.blocked`；实测 **erp-system/ack/** 有 **1804** 文件、**52** 个 **drift-notified marker**（— **sha** 一个、永久）；**worktree** **GC** 存在但手动 **dry-run-default**、非 **launchd/cron** 驱动，孤儿 **worktree** 累积到有人手动跑——这是 **24GB** 的最大贡献者。

6. 并发安全 — ✅ 有（近期硬化，**HEAD** 处补上一处历史缺口）两个互补锁原语 + 原子文件操作 + 内核 **fenced flock**。 `project_spawn_lock`（原子 **mkdir** / **TTL-break** / 封顶 **retry** / **wait budget**）覆盖派窗意图 + 串行化并行-**worktree** **git** 改动，**crash-free** under its own race（输者见赢者 **fresh** 锁 → 干净 **LockHeld**，从未未捕获 **FileExistsError**）。 `acquire_dir_lock`（**flock**）内核 **fenced**、持有者死时自动释放、**re-entrant** via **pid-keyed registry**、**O_CLOEXEC** 不泄进 **git** 子进程、无 **staleness** 启发故根除 **acquire/stale-clear** **TOCTOU**。曾被标记的 `.uri` / **create_worktree** **race** 已在 **HEAD** 关闭——**dump** 的 **worktree** 创建现在跑在 `project_spawn_lock` 下（`dump.py:729`），与 **spawn** 对称。**legacy** **mkdir-era** `*.lockdir` 挡新 **flock** 抛 **LockMigrationError**（运维手动移除）而非 **auto-rmdir**。残留：**alive-but-hung** 持有者不强破；多线程同路径 **flock** 不支持（消费方是单线程 **CLI**，实践安全）。

7. 测试 / **CI** — ✅ 有（强，但覆盖偏 **unit/integration**；**live E2E** 靠人驱动）**67** 个测试文件、**1459** 个 `def test_` 函数（覆盖原子性 / **spawn-lock** 并发 / **worktree** 生命周期 / **succession authority** / 审计闸 **Phase A-D** / **unlock** 路由 / **focus-drift** / **singlepane** / **retro mandate**）。**CI** 真矩阵：**GitHub Actions** on **push+PR** to **main**，**3 OS × 3 Python**（**ubuntu+macos** × **3.11/3.12/3.13**），**pytest** + **console-script smoke** + 冪等 **installer smoke** + 独立 `ruff check` / `ruff format --check` + **sdist/wheel** **build job**（**gated** on **test+lint**）。**ruff** 精确 **pin**（`ruff==0.15.5`，注释说明未 **pin** 漂红了树）。**Gap**：真屏解锁、真跨桌面 **focus-jump**、真 **osascript** **Enter** 在 **CI** 跑不了（无 **GUI** / 无锁）——这些由 **crafted-window** / 人工 **live E2E** 验证，即最高风险行为有最弱的自动覆盖（势所必然），测试 **pin lock=unlocked** / **stub** 路由 **CLI**。

8. 依赖 / 供应链 — ✅ 有（最小、干净——最强维度）**Python** 包零运行时依赖（`dependencies = []`，纯 **stdlib**）；**dev/lint** **extras only**。**VS Code** 扩展零运行时依赖（`dependencies: {}`，全 **devDeps**），`package-lock.json` 在仓，无 **npm runtime** 供应链面发给用户。外部 **CLI**（**shell-out** / **host** 提供 / 未 **pin**）：**osascript** / **codex** / **ioreg** / **caffeinate** / **vscode-spaces.py** / **gemini** / **winlist** + **dual-brain-runner.py** / **code** / **MindPersist** **venv**——这是真正的供应链面，每次调用 **fail-open** 或 **env** 可覆盖（缺失/变更降级非损坏）。**winlist** 与 **vscode-spaces.py** 住 **dhar-maxis**、非 **pip** 安装，是无 **manifest** 捕获的跨仓运行时耦合。

NFR 半实现陷阱小结：① `coord-identity-diag.log` **5MB** 后是静默黑洞——看着可观测、负载下变盲。② “已备份”是 **host** 的幻觉非系统的——**fsync everywhere** 的写持久性掩盖了状态恢复的缺失。③ 审计闸部署臂能 **fail-open-into-no-deploy** 却看着像成功——**commit** 触及 **launcher** 可“成功”而 **runtime** 留旧、只一条 **stderr** **WARN**，叠加 **ff-merge** 不触发 **post-commit**，**live launcher** 能静默滞后已提交源。④ “**unlock** 默认 **OFF**”对机制为真、但 **owner** 已对 **erp-system** 武装——安全预算是花掉了不是留着。⑤ 哨兵累积假装无害——不损坏所以看着没事，但是 **24GB** 树的来源、且 **ack/** 扫描随龄线性变慢、无自动地板。⑥ `docs/ARCHITECTURE.md`（**5-layer defense** 文档）现在只覆盖系统约 **1/3**——零提及 **unlock-pivot**、交付审计机器闸、中枢/**succession authority**、**singlepane**、**per-session worktree**、**winlist** 桌面路由。

6. 当前 release stage + 距「可安全自托管运营」缺口

handoff-fanout 是一套已上线、运营中的内网系统——去程 focus-jump (router.log 70 条直跳)、回程一步重激活 (auto-continue.log 31 条)、watchdog 6-mode backstop + 529 防御。两道“闸”的强制范围须分清 (勿笼统说“都 live 拦截”)：① `pre-push audit-check git hook` 对 handoff-fanout 每次推 main 都拦 (git 钩子级·不受 config 限制·已实测拦/放)；② `dump` 时的 `retro/audit mandate` 受 `config.json:mandate_projects` 限定——当前值 `["erp-system"]`，handoff-fanout 不在内，故其无证据 `dump` 走 `legacy` 路径 (`dump.py:293` 对未列项目不硬拦)，handoff-fanout 的 `dump`-时强制实际来自显式 `--retro-evidence` (`coordinator audit-close` 必传) + `$0` 自检，不是 `mandate_projects` 硬闸；且 `env mandate` 无运行时自证/漂移告警 (见 GAP D)。扩展 0.6.0 已装且单栏折叠 / `$6c reclaim` / `PID dead-man` 接线完整 (`$6c reclaim` 代码 live、回收 E2E 从未成功，见 `$3.5/GAP A4`)。对一个单用户单机工具，它已越过“能跑”进入“日常驱动多会话编排”。

距“可安全自托管运营”还有几道诚实的缺口，按风险排序：

- **install.sh 会卸掉 live 扩展 (最高优先)**：步骤 5 还停在“扩展 obsolete、autoclose 被砍”的旧世界观，主动 `uninstall handoff-helper`，而单栏折叠与 `$6c reclaim` 都建在这个 0.6.0 扩展上。在改回“安装/升级到 0.6.0”之前，跑 `install.sh` 必须 `--no-extension`，否则一次 `install` 就让冷启动窗口失去单栏折叠、`worktree` 永不回收、`succession` 关窗失效。
- **状态树无应用级备份/恢复**：24GB 的 `~/claude-handoff` 不版本控制、无 `export/restore`，只靠 Time-Machine 顺带。目录丢失而 TM 不 `current` 时，在飞 `handoff` / 审计证据 / `succession token` 不可恢复。需要一条状态备份/恢复故事，而非依赖 host TM。
- **哨兵无限累积 + worktree GC 非自动**：`prune.py` 只 GC queue done/blocked, ack/ 1804 文件、drift marker — sha 一个永久、`worktree` GC 手动 `dry-run-default`、`gc-singlepane` 无 `launchd` 周期——累积是 24GB 的主因且让扫描随龄变慢。需要把 GC 纳入定期 (`launchd` 末尾 `dry-run-only` 告警或 `runbook` 登记定期手动 GC)。
- **带凭据的 unlock 路径对一个项目 LIVE**：CGEvent 登录密码注入对 `erp-system` 拨 ON——这是 owner 自觉接受的物理风险，机制安全 (仅 `Keychain` / 默认 off / 三道护栏 / 不下放 `Live-DB-root`)，但“默认 OFF”不等于“凭据注入休眠”，安全预算已对该项目花出。
- **\$6c reclaim 末端到端验证**：`producer` (`watchdog tick`) 接线、代码全活，但盘上当前零回收哨兵 = 此刻无在飞回收。声称端到端 LIVE 须有一次真 `coordinator`→扩展→`PID-ESRCH`→`worktree` 删的证据，仅 `tick` 接线是 `producer` 侧不是 E2E。
- **mandate 生效性靠会话外 env、缺运行时自证**：复盘/审计闸“已拨”只存在于三条 `env` 注入路径，任一漂移则 `dump` 静默走 `legacy` 放行而无告警；闸的存在 ≠ 闸在跑，缺一个开张自检探针。
- **文档大面积滞后实现**：`docs/PROTOCOL.md` (`schema_version 2`)、`docs/ARCHITECTURE.md` (5-layer, 约覆盖 1/3)、被代码引为 SOT 的 `codex-audit-gate-spec-draft.md` (不存在)、多处把已上线链注释成休眠——读文档的人会错判系统范围与门禁生效性。
- **两处确认的并发/挂死 P1 (非数据损坏，但能 wedge 一个项目)**：`project_spawn_lock` 的 TTL stale-break 无 liveness 检查、慢 `create_worktree` (>120s) 会被仍轮询的 worker 误破锁致两个临界区并发改同一 repo；回程 `helper` 无 `wall-clock` 超时、`swiftc` 编译卡死能冻结整个接续 `iteration` (`lock` 不释放 / 后续 `spawn` 全阻塞)。两者都有现成缓解方向 (`held-lock heartbeat` / `PID liveness` / 提高 `ttd`；用既有 `run_with_timeout` 包回程 + 给 `swiftc` 加 `timeout`)。